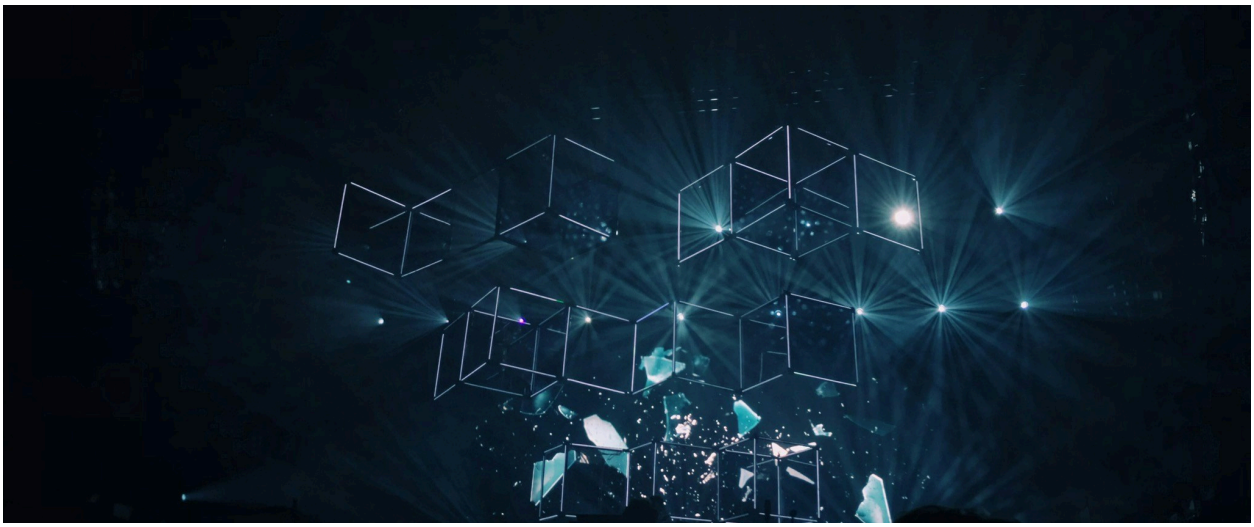

MySQL Database Objects

By Kranti Patil
Fusion Software Institute

📊 MySQL Indexes

Definition

An **index** in MySQL is a data structure that improves the speed of data retrieval operations on a table.

MySQL Index Syntax

```
-- Create Index
CREATE INDEX index_name ON table_name (column1, column2);
-- Drop Index
DROP INDEX index_name ON table_name;
```

MySQL Index Examples

```
-- Create sample table
CREATE TABLE employees (
```

```
    emp_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    email VARCHAR(100),
    department VARCHAR(50),
    salary DECIMAL(10,2),
    hire_date DATE,
    bio TEXT
);

-- Regular index on last_name
CREATE INDEX idx_lastname ON employees(last_name);

-- Composite index
CREATE INDEX idx_dept_salary ON employees(department, salary);

-- Unique index on email
CREATE UNIQUE INDEX idx_email ON employees(email);

-- Show indexes on table
SHOW INDEX FROM employees;

-- Analyze index usage
EXPLAIN SELECT * FROM employees WHERE last_name = 'Smith';
```

🔧 MySQL Stored Procedures

Definition

A **stored procedure** in MySQL is a prepared SQL code that can be saved and reused. MySQL stored procedures support parameters, variables, conditions, and loops.

MySQL Stored Procedure Syntax

```
DELIMITER //
CREATE PROCEDURE procedure_name(
    IN parameter1 datatype,
    OUT parameter2 datatype,
    INOUT parameter3 datatype
)
BEGIN
    -- Procedure body
    DECLARE variable_name datatype DEFAULT value;
    -- SQL statements
END //
DELIMITER ;

-- Call stored procedure
CALL procedure_name(value1, @output_var, value3);
```

MySQL Stored Procedure Examples

```
-- Simple procedure with IN parameter
DELIMITER //
CREATE PROCEDURE GetEmployeesByDepartment(
    IN dept_name VARCHAR(50)
)
BEGIN
    SELECT emp_id, first_name, last_name, salary
    FROM employees
    WHERE department = dept_name
    ORDER BY salary DESC;
END //
DELIMITER ;

-- Call the procedure
CALL GetEmployeesByDepartment('Engineering');
```

```sql
-- Procedure with IN, OUT, and INOUT parameters
DELIMITER //
CREATE PROCEDURE EmployeeStatistics(
    IN dept_name VARCHAR(50),
    OUT emp_count INT,
    OUT avg_salary DECIMAL(10,2),
    INOUT min_salary DECIMAL(10,2)
)
BEGIN
    SELECT
        COUNT(*),
        AVG(salary),
        MIN(salary)
    INTO emp_count, avg_salary, min_salary
    FROM employees
    WHERE department = dept_name
      AND salary >= min_salary;
END //
DELIMITER ;

-- Call with variables
SET @count = 0;
SET @avg_sal = 0;
SET @min_sal = 50000;
CALL EmployeeStatistics('Sales', @count, @avg_sal, @min_sal);
SELECT @count AS employee_count, @avg_sal AS average_salary, @min_sal
AS minimum_salary;
```

👁️ MySQL Views

Definition

A **view** in MySQL is a virtual table based on the result of an SQL statement. Views in MySQL are stored queries that can be treated like tables but don't store data

themselves.

MySQL View Types

- **Simple Views**: Based on a single table
- **Complex Views**: Based on multiple tables with JOINs

MySQL View Syntax

```
-- Create View
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;

-- Drop View
DROP VIEW IF EXISTS view_name;
```

MySQL View Examples

```
-- Create departments table for examples
CREATE TABLE departments (
    dept_id INT AUTO_INCREMENT PRIMARY KEY,
    dept_name VARCHAR(50) UNIQUE,
    manager_id INT,
    budget DECIMAL(12,2)
);

-- Insert sample data
INSERT INTO departments (dept_name, manager_id, budget) VALUES
('Engineering', 101, 1000000),
('Sales', 102, 500000),
('Marketing', 103, 300000),
('HR', 104, 200000);

-- Simple view: High salary employees
```

```sql
CREATE VIEW high_salary_employees AS
SELECT
    emp_id,
    CONCAT(first_name, ' ', last_name) AS full_name,
    email,
    department,
    salary,
    ROUND(salary * 12, 2) AS annual_salary
FROM employees
WHERE salary > 75000;

-- Query the view
SELECT * FROM high_salary_employees WHERE department = 'Engineering';

-- Complex view with JOIN
CREATE VIEW employee_department_details AS
SELECT
    e.emp_id,
    e.first_name,
    e.last_name,
    e.email,
    e.salary,
    d.dept_name,
    d.budget AS dept_budget,
    ROUND((e.salary / d.budget) * 100, 2) AS salary_budget_percentage
FROM employees e
JOIN departments d ON e.department = d.dept_name;

-- View with aggregation
CREATE VIEW department_summary AS
SELECT
    department,
    COUNT(*) AS employee_count,
    AVG(salary) AS avg_salary,
    MIN(salary) AS min_salary,
    MAX(salary) AS max_salary,
```

```
    SUM(salary) AS total_payroll
FROM employees
GROUP BY department;

-- Show all views
SHOW FULL TABLES WHERE Table_type = 'VIEW';
```

⚡ MySQL Triggers

Definition

A **trigger** in MySQL is a stored program that automatically executes (fires) in response to specific events on a table or view in a database.

MySQL Trigger Types

- **BEFORE**: Executes before the triggering event
- **AFTER**: Executes after the triggering event

MySQL Trigger Events

- **INSERT**: When new rows are added
- **UPDATE**: When existing rows are modified
- **DELETE**: When rows are removed

MySQL Trigger Syntax

```
DELIMITER //
CREATE TRIGGER trigger_name
{BEFORE | AFTER} {INSERT | UPDATE | DELETE}
ON table_name
FOR EACH ROW
BEGIN
    -- Trigger logic
    -- Access OLD values with OLD.column_name
    -- Access NEW values with NEW.column_name
```

```
END //
DELIMITER ;
```

MySQL Trigger Examples

```
-- Create audit table
CREATE TABLE employee_audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    emp_id INT,
    action_type ENUM('INSERT', 'UPDATE', 'DELETE'),
    old_salary DECIMAL(10,2),
    new_salary DECIMAL(10,2),
    old_department VARCHAR(50),
    new_department VARCHAR(50),
    changed_by VARCHAR(100),
    change_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- BEFORE INSERT trigger for validation
DELIMITER //
CREATE TRIGGER before_employee_insert
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
    -- Validate salary range
    IF NEW.salary < 30000 OR NEW.salary > 200000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary must be between 30000 and 200000';
    END IF;

    -- Validate email format
    IF NEW.email NOT REGEXP '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]
{2,}$' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Invalid email format';
```

```
    END IF;

    -- Auto-set hire_date if not provided
    IF NEW.hire_date IS NULL THEN
        SET NEW.hire_date = CURDATE();
    END IF;
END //
DELIMITER ;

-- AFTER INSERT trigger for audit
DELIMITER //
CREATE TRIGGER after_employee_insert
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employee_audit (
        emp_id,
        action_type,
        new_salary,
        new_department,
        changed_by
    ) VALUES (
        NEW.emp_id,
        'INSERT',
        NEW.salary,
        NEW.department,
        USER()
    );
END //
DELIMITER ;

-- BEFORE UPDATE trigger
DELIMITER //
CREATE TRIGGER before_employee_update
BEFORE UPDATE ON employees
FOR EACH ROW
```

```
BEGIN
    -- Prevent salary decrease of more than 20%
    IF NEW.salary < OLD.salary * 0.8 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be decreased by more than 20%';
    END IF;

    -- Prevent department change for employees with less than 6 months tenur
e
    IF NEW.department != OLD.department AND
       DATEDIFF(CURDATE(), OLD.hire_date) < 180 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot change department within 6 months of hi
re';
    END IF;
END //
DELIMITER ;

-- AFTER UPDATE trigger for audit
DELIMITER //
CREATE TRIGGER after_employee_update
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
    -- Only log if salary or department changed
    IF OLD.salary != NEW.salary OR OLD.department != NEW.department THEN
        INSERT INTO employee_audit (
            emp_id,
            action_type,
            old_salary,
            new_salary,
            old_department,
            new_department,
            changed_by
        ) VALUES (
            NEW.emp_id,
```

```
            'UPDATE',
            OLD.salary,
            NEW.salary,
            OLD.department,
            NEW.department,
            USER()
        );
    END IF;
END //
DELIMITER ;

-- AFTER DELETE trigger
DELIMITER //
CREATE TRIGGER after_employee_delete
AFTER DELETE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employee_audit (
        emp_id,
        action_type,
        old_salary,
        old_department,
        changed_by
    ) VALUES (
        OLD.emp_id,
        'DELETE',
        OLD.salary,
        OLD.department,
        USER()
    );
END //
DELIMITER ;

-- Test the triggers
INSERT INTO employees (first_name, last_name, email, department, salary)
VALUES ('John', 'Doe', 'john.doe@company.com', 'Engineering', 80000);
```

```
UPDATE employees SET salary = 85000 WHERE emp_id = 1;

DELETE FROM employees WHERE emp_id = 1;

-- Check audit trail
SELECT * FROM employee_audit ORDER BY change_timestamp DESC;

-- Show all triggers
SHOW TRIGGERS;

-- Show triggers for specific table
SHOW TRIGGERS WHERE `Table` = 'employees';

-- Drop trigger
DROP TRIGGER IF EXISTS trigger_name;
```

MySQL Trigger Limitations

- Cannot call stored procedures that return result sets
- Cannot use explicit or implicit COMMIT or ROLLBACK
- Cannot return values to the calling program
- Only one trigger per event per table (before MySQL 5.7.2)